V-OS

Γραφικό λειτουργικό σύστημα για τον Amstrad CPC 6128

Τεκμηρίωση

Φτιαγμένο από τα docs/manual/*.md και το sdk/README.md με το tools/mkdocs.py (make docs).

Περιεχόμενα

Το V-OS είναι γραφικό λειτουργικό σύστημα για τον Amstrad CPC 6128, γραμμένο από την αρχή σε Z80 assembly. Έχει:

- γραμμή μενού, παράθυρα που σέρνονται, διαλόγους και κουμπιά, με πληκτρολόγιο ή ποντίκι AMX
- τρεις όψεις για αρχεία και προγράμματα (Desktop, Launcher, Commander) και τις ρυθμίσεις (Setup)
- έξι γλώσσες ως plugins: αγγλικά, ελληνικά, γαλλικά, ισπανικά, γερμανικά και ιταλικά
- προγράμματα που τρέχουν δίπλα-δίπλα και συνεχίζουν όπου σταμάτησαν, ένα desk accessory και προγράμματα στο παρασκήνιο
- το Studio, έναν editor blocks τύπου Scratch που τρέχει πάνω στον ίδιο τον CPC
- ένα API με 42 κλήσεις και SDK για προγράμματα σε assembly

- 1. Ξεκίνημα
- 2. Οι όψεις
- 3. Προγράμματα
- 4. Γλώσσες
- 5. Studio: προγράμματα με blocks
- 6. Blocks στον υπολογιστή: ο web editor και το `vbc.py`
- 7. SDK του V-OS: προγράμματα σε assembly
- 8. Ανάπτυξη του V-OS

1. Ξεκίνημα

Τι χρειάζεσαι

- Έναν Amstrad CPC 6128 ή έναν εξομοιωτή του. Το V-OS ελέγχεται στον Caprice32.
- Τη δισκέτα του V-OS στο drive A:, δηλαδή το `build/vos.dsk` που φτιάχνει το `make`.
- Προαιρετικά:
  - ποντίκι AMX
  - δεύτερο drive (B:)
  - επέκταση μνήμης (π.χ. 512K), για περισσότερα προγράμματα μαζί

Εκκίνηση

Από τη BASIC:

```
RUN"VOS
```

Το V-OS φορτώνει τον kernel, κλείνει τις ROM και αναλαμβάνει όλο το μηχάνημα. Ανιχνεύει μόνο του την επέκταση μνήμης.

- **Πρώτη όψη:** το Desktop (ΕΠΙΦΑΝΕΙΑ). Αλλάζει στο Setup (ΡΥΘΜΙΣΕΙΣ), στη γραμμή ΟΨΗ ΕΚΚΙΝΗΣΗΣ.
- **Γλώσσα:** αγγλικά. Αν υπάρχει `SETUP.CFG` στη δισκέτα, ξεκινά με τη γλώσσα και τις ρυθμίσεις που αποθηκεύτηκαν εκεί.
- **Έξοδος:** ΑΡΧΕΙΟ > ΕΞΟΔΟΣ ΣΤΗ BASIC (CONTROL+Q στις όψεις). Ζητά επιβεβαίωση και ξαναξεκινά τον CPC.

Η οθόνη

Το V-OS χρησιμοποιεί όλη την οθόνη (overscan): 384×272 pixels σε mode 1 (4 χρώματα) ή 192×272 σε mode 0 (16 χρώματα).

- **Γραμμή μενού** (πάνω): τα μενού της όψης ή του προγράμματος, συνήθως ΑΡΧΕΙΟ ή το μενού του προγράμματος, ΠΡΟΒΟΛΗ και ΒΟΗΘΕΙΑ. Δεξιά φαίνεται η γλώσσα του πληκτρολογίου (EN, ΕΛ, ...).
- **Κάτω γραμμή:** σε κάθε όψη, τα πλήκτρα που ισχύουν εκεί.

Πλήκτρα που ισχύουν παντού

Πλήκτρο	Τι κάνει
F1	Desktop (ΕΠΙΦΑΝΕΙΑ)
F2	Launcher (ΠΡΟΓΡΑΜΜΑΤΑ)
F3	Commander
F4	Setup (ΡΥΘΜΙΣΕΙΣ)
F5	ανοίγει και κλείνει το desk accessory (ΑΞΕΣΟΥΑΡ)
F6	τα προγράμματα που τρέχουν, για να διαλέξεις ένα
ESC	ανοίγει το πρώτο μενού. Στα μενού, ←/→ αλλάζουν μενού, ↑/↓ επιλογή, RETURN την εκτελεί, ESC κλείνει

CONTROL+γράμμα	οι συντομεύσεις των μενού (π.χ. ^Q, ^L)
CONTROL+L	εναλλαγή γλώσσας (ΠΡΟΒΟΛΗ > ΑΛΛΑΓΗ ΓΛΩΣΣΑΣ)
CONTROL+K	ο διάλογος με τα πλήκτρα (ΒΟΗΘΕΙΑ > ΠΛΗΚΤΡΑ)
CONTROL+SHIFT	αλλάζει το πληκτρολόγιο ανάμεσα στα αγγλικά και τη γλώσσα του pack
TAB, SHIFT+TAB	μετακινούν την εστίαση στα κουμπιά. RETURN, ENTER ή SPACE πατούν το κουμπί που έχει την εστίαση

Τα F1–F6 είναι τα πλήκτρα f1–f6 του αριθμητικού πληκτρολογίου του CPC.

Ο δείκτης και το ποντίκι

- **Με ποντίκι AMX:** ο δείκτης κινείται με το ποντίκι, και το κουμπί του κάνει κλικ. Το ποντίκι ενεργοποιείται και ρυθμίζεται στο Setup (ΠΟΝΤΙΚΙ AMX, ΑΝΑΓΝΩΣΗ AMX).
- **Χωρίς ποντίκι:** ο δείκτης κινείται με τα βελάκια, και το COPY κάνει κλικ. Σε όψεις που χρησιμοποιούν τα βελάκια για επιλογή (Desktop, Launcher, Commander), τα βελάκια επιλέγουν αρχεία και προγράμματα αντί να κινούν τον δείκτη.

Τα περισσότερα στοιχεία δουλεύουν και με τα δύο: ένα κλικ σε αρχείο ή πρόγραμμα το επιλέγει, και δεύτερο κλικ το ανοίγει.

Παράθυρα και διάλογοι

- **Σύρσιμο:** κάθε παράθυρο σέρνεται από τη γραμμή τίτλου του: οι διάλογοι, το παράθυρο ΑΡΧΕΙΑ του Desktop, το desk accessory, και τα παράθυρα των προγραμμάτων με blocks. Όσο κρατάς το κουμπί φαίνεται ένα σπαστό περίγραμμα, και το παράθυρο ξανασχεδιάζεται εκεί που το αφήνεις.
- **Διάλογοι:** περιμένουν απάντηση. Το TAB και τα ←/→ αλλάζουν κουμπί, το RETURN το πατά, και το ESC διαλέγει το τελευταίο (συνήθως ΟΧΙ ή ΑΚΥΡΟ).
- **Εισαγωγή κειμένου** (π.χ. μετονομασία): γράφεις στο τέλος, το DEL σβήνει, το RETURN δέχεται και το ESC ακυρώνει. Γράφεις και στη γλώσσα του pack.

Δισκέτες και αρχεία

- **Μορφή:** το V-OS διαβάζει και γράφει δισκέτες AMSDOS (μορφή DATA ή SYSTEM) στα A: και B:. Τα αρχεία του τα διαβάζει και η BASIC.
- **Ονόματα:** 8 χαρακτήρες και 3 για τον τύπο (`ΟΝΟΜΑ.ΤΥΠ`).

Τύποι που συναντάς:

Τύπος	Τι είναι
`.BIN`	πρόγραμμα του V-OS (ή αρχείο binary)
`.VBN`	πρόγραμμα του V-OS φτιαγμένο με blocks
`.VBS`	πρόγραμμα του Studio (η πηγή του)
`.OVL`, `.MSG`, `.LNG`	κομμάτια του ίδιου του V-OS: όψεις, κείμενα, γλώσσες
`.SYS`	αρχεία συστήματος (π.χ. `VBRUN.SYS`, ο παίκτης του Studio)
`SETUP.CFG`	οι ρυθμίσεις

Οι ρυθμίσεις, οι γλώσσες και οι όψεις διαβάζονται από το A:, οπότε η δισκέτα του V-OS πρέπει να μένει εκεί.

Μηνύματα σφάλματος

Τα σφάλματα του δίσκου φαίνονται σε διάλογο: ΔΕΝ ΕΙΝΑΙ ΕΤΟΙΜΗ (το drive), ΠΡΟΣΤΑΣΙΑ ΕΓΓΡΑΦΗΣ, ΔΕΝ ΒΡΕΘΗΚΕ (το αρχείο), ΓΕΜΑΤΗ ΔΙΣΚΕΤΑ, ΓΕΜΑΤΟΣ ΚΑΤΑΛΟΓΟΣ, ΤΟ ΟΝΟΜΑ ΥΠΑΡΧΕΙ, ΜΗ ΕΓΚΥΡΟ ΟΝΟΜΑ κ.ά.

Δύο ακόμα αφορούν προγράμματα:

- **ΔΕΝ ΕΙΝΑΙ ΠΡΟΓΡΑΜΜΑ V-OS:** το αρχείο δεν έχει την κεφαλίδα ενός προγράμματος.
- **ΚΛΕΙΣΤΕ ΠΡΩΤΑ ΑΛΛΟ ΠΡΟΓΡΑΜΜΑ:** δεν υπάρχει χώρος. Βλ. κεφάλαιο 3.

2. Οι όψεις

Το V-OS έχει τέσσερις «όψεις», δηλαδή τρόπους να δεις τα αρχεία και τα προγράμματα, και τις ρυθμίσεις. Αλλάζεις όψη με τα F1–F4 ή από το μενού ΠΡΟΒΟΛΗ. Η κάθε όψη θυμάται ποιο αρχείο ήταν επιλεγμένο, και η επόμενη επιλέγει το ίδιο.

Τα κοινά μενού

Τα δύο τελευταία μενού είναι ίδια σε όλες τις όψεις και στα προγράμματα:

ΠΡΟΒΟΛΗ		**ΒΟΗΘΕΙΑ**	
ΕΠΙΦΑΝΕΙΑ	F1	ΠΛΗΚΤΡΑ	^K
ΠΡΟΓΡΑΜΜΑΤΑ (Launcher)	F2	ΣΧΕΤΙΚΑ ΜΕ ΤΟ V-OS	
COMMANDER	F3		
ΡΥΘΜΙΣΕΙΣ	F4		
ΠΡΟΓΡΑΜΜΑΤΑ (που τρέχουν)	F6		
ΑΛΛΑΓΗ ΓΛΩΣΣΑΣ	^L		

Desktop (ΕΠΙΦΑΝΕΙΑ), F1

Mode 1. Ένα παράθυρο ΑΡΧΕΙΑ δείχνει τα αρχεία ενός drive ως εικονίδια, τέσσερα σε κάθε σειρά. Ο τύπος γράφεται μέσα στο εικονίδιο και το όνομα από κάτω, ενώ ένα `.VBN` έχει δίπλα ένα σημάδι «play». Δεξιά, δύο εικονίδια διαλέγουν το A: ή το B:. Στη γραμμή τίτλου φαίνονται η μορφή της δισκέτας και το πλήθος των αρχείων, και κάτω ο ελεύθερος χώρος.

Πλήκτρο	**Τι κάνει**
βελάκια	επιλογή αρχείου (↑/↓ μία σειρά)
TAB	το άλλο drive
RETURN, ENTER	στοιχεία του αρχείου
^C	αντιγραφή στο άλλο drive
^D	διαγραφή (ρωτά πρώτα)
^N	μετονομασία
^R	ξαναδιάβασμα των δισκετών
ποντίκι	κλικ σε αρχείο: επιλογή, ξανά: στοιχεία. Κλικ σε drive: το ανοίγει

- **Μενού ΑΡΧΕΙΟ:** ΕΚΚΙΝΗΣΗ (τρέχει το επιλεγμένο πρόγραμμα), ΣΤΟΙΧΕΙΑ ΑΡΧΕΙΟΥ, ΑΝΤΙΓΡΑΦΗ ΣΤΗΝ ΑΛΛΗ ΜΟΝΑΔΑ, ΔΙΑΓΡΑΦΗ, ΜΕΤΟΝΟΜΑΣΙΑ, ΞΑΝΑΔΙΑΒΑΣΜΑ ΔΙΣΚΩΝ, ΕΞΟΔΟΣ ΣΤΗ BASIC.
- **Το παράθυρο ΑΡΧΕΙΑ** σέρνεται από τον τίτλο του και μένει εκεί, ακόμα κι όταν αλλάζεις όψη.

Launcher (ΠΡΟΓΡΑΜΜΑΤΑ), F2

Mode 0, 16 χρώματα. Δείχνει μόνο τα προγράμματα του A: (`.BIN`, `.VBN`, `.BAS`), ως πλακίδια σε πλέγμα 3×3, με τις επόμενες σελίδες από κάτω. Κάθε πρόγραμμα που έχει εικονίδιο το δείχνει στο πλακίδιό του. Το επιλεγμένο πλακίδιο είναι άσπρο.

Πλήκτρο	**Τι κάνει**
βελάκια	επιλογή (↑/↓ μία σειρά. Μετά την άκρη, η επόμενη ή η προηγούμενη σελίδα)
RETURN, ENTER	τρέχει το πρόγραμμα. Αν τρέχει ήδη, συνεχίζει από εκεί που ήταν

^A	το φορτώνει ως desk accessory (ΩΣ ΑΞΕΣΟΥΑΡ). Το F5 το ανοίγει μετά
^R	ξαναδιάβασμα της δισκέτας
ποντίκι	κλικ: επιλογή, ξανά: εκκίνηση

Μενού ΑΡΧΕΙΟ: ΕΚΚΙΝΗΣΗ, ΩΣ ΑΞΕΣΟΥΑΡ, ΞΑΝΑΔΙΑΒΑΣΜΑ ΔΙΣΚΩΝ, ΕΞΟΔΟΣ ΣΤΗ BASIC.

Commander, F3

Mode 1, δύο πάνελ με λίστες αρχείων: αριστερά το A:, δεξιά το B:. Κάθε πάνελ δείχνει στον τίτλο του τη μορφή και το πλήθος των αρχείων, και κάτω τον ελεύθερο χώρο. Το ενεργό πάνελ έχει σκούρο τίτλο και επιλογή.

Πλήκτρο	Τι κάνει
↑/↓	επιλογή (με SHIFT: μία σελίδα)
TAB, ←/→	το άλλο πάνελ
RETURN, ENTER	στοιχεία: μέγεθος, τύπος, διεύθυνση φόρτωσης, τα πρώτα bytes
^C	αντιγραφή στο άλλο πάνελ, οποιουδήποτε μεγέθους. Αν υπάρχει αρχείο με το ίδιο όνομα, ρωτά πρώτα
^D	διαγραφή
^N	μετονομασία
^R	ξαναδιάβασμα και των δύο δισκετών
ποντίκι	κλικ: επιλογή, ξανά: στοιχεία

Μενού ΑΡΧΕΙΟ: ΕΚΚΙΝΗΣΗ, ΣΤΟΙΧΕΙΑ ΑΡΧΕΙΟΥ, ΑΝΤΙΓΡΑΦΗ ΑΠΕΝΑΝΤΙ, ΔΙΑΓΡΑΦΗ, ΜΕΤΟΝΟΜΑΣΙΑ, ΞΑΝΑΔΙΑΒΑΣΜΑ ΔΙΣΚΩΝ, ΦΟΡΜΑΡΙΣΜΑ ΔΙΣΚΕΤΑΣ (DATA), ΕΞΟΔΟΣ ΣΤΗ BASIC.

> Το ΦΟΡΜΑΡΙΣΜΑ σβήνει όλα τα αρχεία της δισκέτας του ενεργού πάνελ. Ρωτά πρώτα.

Setup (ΡΥΘΜΙΣΕΙΣ), F4

Mode 1. Μία γραμμή για κάθε ρύθμιση. Οι αλλαγές ισχύουν αμέσως. Με αποθήκευση (RETURN ή ^S) γράφονται στο `A:SETUP.CFG` και ισχύουν και στην επόμενη εκκίνηση.

Ρύθμιση	Τιμές
ΟΨΗ ΕΚΚΙΝΗΣΗΣ	ΕΠΙΦΑΝΕΙΑ, ΠΡΟΓΡΑΜΜΑΤΑ, COMMANDER
ΓΛΩΣΣΑ	ENGLISH, ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL, DEUTSCH, ITALIANO
ΚΑΘΥΣΤ. ΕΠΑΝΑΛΗΨΗΣ	πόσο κρατάς ένα πλήκτρο ώσπου να αρχίσει να επαναλαμβάνεται: ΣΥΝΤΟΜΗ, ΜΕΣΑΙΑ, ΜΕΓΑΛΗ
ΤΑΧΥΤ. ΕΠΑΝΑΛΗΨΗΣ	ΑΡΓΗ, ΜΕΣΑΙΑ, ΓΡΗΓΟΡΗ
ΠΟΝΤΙΚΙ AMX	ΝΑΙ, ΟΧΙ
ΑΝΑΓΝΩΣΗ AMX	ΠΑΛΜΟΙ ή ΣΤΑΘΜΕΣ: πώς διαβάζεται το ποντίκι. Αν ο δείκτης δεν κινείται σωστά, δοκίμασε το άλλο
ΧΡΩΜΑ ΦΟΝΤΟΥ	ΜΠΛΕ, ΜΑΥΡΟ, ΠΡΑΣΙΝΟ, ΚΟΚΚΙΝΟ

Πλήκτρο	Τι κάνει
↑/↓	επιλογή ρύθμισης
←/→	αλλαγή τιμής (και κλικ στη γραμμή)
RETURN, ^S	αποθήκευση

Η αλλαγή ΓΛΩΣΣΑΣ φορτώνει αμέσως το pack της γλώσσας από το A:. Βλ. κεφάλαιο 4.

3. Προγράμματα

Εκκίνηση και τέλος

Ένα πρόγραμμα ξεκινά με έναν από αυτούς τους τρόπους:

- από τον Launcher: επιλογή και RETURN, ή δύο κλικ
- από το Desktop ή τον Commander: ΑΡΧΕΙΟ > ΕΚΚΙΝΗΣΗ
- από άλλο πρόγραμμα, όπως το Studio που τρέχει ό,τι φτιάχνεις

Κάθε πρόγραμμα έχει δικό του μενού δίπλα στα κοινά ΠΡΟΒΟΛΗ και ΒΟΗΘΕΙΑ. Τελειώνει από το μενού του (συνήθως ΕΠΙΣΤΡΟΦΗ ή ΤΕΛΟΣ, CONTROL+Q) και γυρίζει στην όψη από την οποία ξεκίνησε.

Πολλά προγράμματα μαζί

Τα προγράμματα του V-OS είναι δύο ειδών:

- **Μικρά, με πίνακα relocation:** τα DEMO, MIXED και CLOCK, τα παραδείγματα του web editor (PONG, CATCH, STARCAT, COUNTER), και όσα φτιάχνει το SDK. Μοιράζονται 12K μνήμης και τρέχουν όσα χωράνε μαζί.
- **Προγράμματα που θέλουν όλη τη μνήμη:** το STUDIO και όσα φτιάχνει το Studio (CALC, MINES, FLAP, TICKER και τα δικά σου). Χωρίς επέκταση μνήμης τρέχουν μόνα τους. Αν τρέχει ήδη κάποιο, βγαίνει «ΚΛΕΙΣΤΕ ΠΡΩΤΑ ΑΛΛΟ ΠΡΟΓΡΑΜΜΑ».

Όταν πας σε μια όψη (F1–F4) ενώ τρέχει ένα πρόγραμμα:

- Ένα μικρό πρόγραμμα μένει στη μνήμη. Το ξαναπιάνεις από τον Launcher (RETURN πάνω του) ή με το F6, και συνεχίζει ακριβώς από εκεί που ήταν.
- Ένα πρόγραμμα που έχει όλη τη μνήμη τελειώνει, εκτός αν υπάρχει επέκταση μνήμης.

Το F6 (ΠΡΟΒΟΛΗ > ΠΡΟΓΡΑΜΜΑΤΑ) δείχνει τη λίστα με τα προγράμματα που τρέχουν. Διαλέγεις ένα με τα βελάκια και RETURN. Αν δεν τρέχει κανένα, βγαίνει «ΚΑΝΕΝΑ ΠΡΟΓΡΑΜΜΑ ΔΕΝ ΤΡΕΧΕΙ».

Με επέκταση μνήμης (π.χ. 512K, που το V-OS βρίσκει μόνο του στην εκκίνηση), κάθε πρόγραμμα που θέλει όλη τη μνήμη παίρνει δική του bank. Έτσι το STUDIO, η CALC, το DEMO και το PONG μπορούν να τρέχουν όλα μαζί.

Στο παρασκήνιο

Ένα πρόγραμμα πίσω από άλλο, ή πίσω από μια όψη, κανονικά **παγώνει** και συνεχίζει όταν ξανάρθει μπροστά.

Κάποια προγράμματα ζητούν να **συνεχίζουν να τρέχουν** και από πίσω, π.χ. το TICKER, που μετρά τον χρόνο. Εκεί δεν σχεδιάζουν και δεν παίζουν ήχο. Όταν ξανάρθουν μπροστά, έχουν μετρήσει και τον χρόνο που έλειπαν.

Στο Studio το επιλέγεις για το πρόγραμμά σου με STUDIO > ΣΤΟ ΠΑΡΑΣΚΗΝΙΟ.

Το desk accessory (ΑΞΕΣΟΥΑΡ)

Ένα μικρό πρόγραμμα, όπως το ΡΟΛΟΙ (CLOCK), μπορεί να ανοίγει σε παράθυρο **πάνω από οτιδήποτε**: όψη ή πρόγραμμα.

1. Στον Launcher, επίλεξε το CLOCK και πάτησε ΑΡΧΕΙΟ > ΩΣ ΑΞΕΣΟΥΑΡ (^A).
2. Από εκεί και πέρα, το **F5** το ανοίγει και το κλείνει, από όπου κι αν είσαι. Το κλείνει και το ESC.

Όσο είναι ανοιχτό, παίρνει αυτό τα πλήκτρα και τα κλικ. Το παράθυρό του σέρνεται από τον τίτλο του. Όταν κλείνει, ό,τι ήταν από κάτω ξανασχεδιάζεται.

Τα προγράμματα της δισκέτας

Πρόγραμμα	Τι είναι
STUDIO	ο editor blocks: φτιάχνεις προγράμματα πάνω στον CPC (κεφάλαιο 5)
DEMO	το πρώτο πρόγραμμα του V-OS: ένα τετράγωνο που κινείται μόνο του. Τα βελάκια το κατευθύνουν και το SPACE αλλάζει χρώμα
MIXED	η «μικτή οθόνη»: μια ζώνη με 16 χρώματα (mode 0) ανάμεσα σε ζώνες mode 1. Ένα κλικ δείχνει τις συντεταγμένες
CLOCK (ΡΟΛΟΙ)	ο χρόνος από την εκκίνηση του V-OS. Τρέχει και ως desk accessory
CALC	αριθμομηχανή σε παράθυρο με κουμπιά, φτιαγμένη στο Studio (υπάρχει και το `CALC.VBS`)
MINES	ναρκαλιευτής 8×6 με 8 νάρκες, φτιαγμένος στο Studio. Κλικ ανοίγει κελί, και το NEW ξεκινά νέο παιχνίδι
BRICKS	σπάσιμο τούβλων: ρακέτα με ←/→, 32 τούβλα, 3 ζωές, επίπεδα με όλο και γρηγορότερη μπάλα. Στο GAME OVER, SPACE για νέο παιχνίδι
VROUVES	tower defence, από το LET'S GO FOR VROUVES του CPC: 7 πίστες, από το Καφενείο ως το νεκροταφείο, με 10 κύματα η καθεμιά. Στη δισκέτα του V-OS είναι η πίστα 1. Όλες είναι στη δική του δισκέτα, `vrouves.dsk`: στο B:, και ΑΡΧΕΙΟ > ΕΚΚΙΝΗΣΗ στον Commander. Βελάκια για τον κέρσορα, 1–6 αμυντικός, SPACE χτίζει, S πουλά, N το επόμενο κύμα, P παύση. Γραμμένο με το SDK (`myapps/vrouves`)
PONG	μπάλα και ρακέτα με τα βελάκια, σκορ και ζωές
CATCH	πιάσε τα αστέρια που πέφτουν
STARCAT	μια γάτα με κοστούμια που περπατά, ένα αστέρι, μελωδία από λίστα
COUNTER	μετρητής με κουμπιά +1, −1 και 0, σε mode 1
FLAP	ένα πουλί που χτυπά τα φτερά του (κοστούμια), κι ένα αστέρι που αλλάζει κοστούμι κάθε δευτερόλεπτο
TICKER	μετρά δευτερόλεπτα και συνεχίζει στο παρασκήνιο

- Τα προγράμματα με blocks είναι αρχεία `.VBN`. Έχουν μενού ΠΡΟΓΡΑΜΜΑ > ΤΕΛΟΣ (CONTROL+Q).
- Όσα έχουν και `.VBS` (CALC, MINES, FLAP, TICKER) ανοίγουν στο Studio για να τα δεις ή να τα αλλάξεις.
- Το `VBRUN.SYS` δεν είναι πρόγραμμα: είναι ο «παίκτης» που χρειάζεται το Studio.

4. Γλώσσες

Το V-OS μιλά έξι γλώσσες. Τα αγγλικά είναι ενσωματωμένα. Οι άλλες πέντε είναι **language packs**, αρχεία στη δισκέτα που φέρνουν τρία πράγματα: όλα τα κείμενα του συστήματος, το πληκτρολόγιο και τα γράμματα (font) της γλώσσας.

Γλώσσα	Αρχείο	Πληκτρολόγιο
English	ενσωματωμένη (`ENGLISH.MSG`)	το αγγλικό του CPC
Ελληνικά	`GREEK.LNG`	ελληνικό, με τόνους
Français	`FRENCH.LNG`	AZERTY, τα ψηφία με SHIFT, νεκρά πλήκτρα ^ και ¨
Español	`SPANISH.LNG`	Ñ στο πλήκτρο `:`, νεκρά πλήκτρα ´ ¨ ` ^, και ¡ ¿
Deutsch	`GERMAN.LNG`	QWERTZ, Ü Ö Ä ß, νεκρά πλήκτρα ´ και `
Italiano	`ITALIAN.LNG`	è é ò à ù ì στα δικά τους πλήκτρα

Αλλαγή γλώσσας

- **Μόνιμα:** στο Setup (F4), η γραμμή ΓΛΩΣΣΑ περνά κυκλικά από όλες τις γλώσσες. Η καθεμία φορτώνεται αμέσως από το A:, και όλη η οθόνη ξαναγράφεται σε αυτήν. Με αποθήκευση (^S) την κρατά και στην επόμενη εκκίνηση.
- **Γρήγορα:** το CONTROL+L (ΠΡΟΒΟΛΗ > ΑΛΛΑΓΗ ΓΛΩΣΣΑΣ) εναλλάσσει ανάμεσα στα αγγλικά και τη γλώσσα του pack που είναι φορτωμένο.
- **Μόνο το πληκτρολόγιο:** το CONTROL+SHIFT εναλλάσσει το πληκτρολόγιο ανάμεσα στο αγγλικό και αυτό του pack. Στη γραμμή μενού, δεξιά, φαίνεται ποιο ισχύει (EN, ΕΛ, FR, ...).

Τα σύμβολα `@ # [ ] { } \ |` δεν υπάρχουν στα πληκτρολόγια των packs, γιατί ο CPC δεν έχει AltGr. Γράφονται με το αγγλικό πληκτρολόγιο.

Τα κείμενα των προγραμμάτων

Τα προγράμματα έχουν τα δικά τους κείμενα σε **δύο γλώσσες**: αγγλικά και, προαιρετικά, μία ακόμα (σήμερα τα ελληνικά). Ένα πρόγραμμα εμφανίζεται στη δεύτερη γλώσσα του μόνο όταν το pack είναι αυτής της γλώσσας. Αλλιώς εμφανίζεται στα αγγλικά. Έτσι, με γαλλικά, το Studio εμφανίζεται στα αγγλικά.

Φτιάχνοντας νέα γλώσσα

Οι πηγές των γλωσσών είναι στο `lang/<κωδικός>/`:

- `messages.txt`: τα κείμενα του συστήματος. Τη βασική λίστα την ορίζει το `lang/en/messages.txt`, και ό,τι λείπει από ένα pack εμφανίζεται στα αγγλικά.
- `keymap.txt`: τα πλήκτρα που αλλάζουν σε σχέση με το αγγλικό.
- `pack.txt`: ο κωδικός, το όνομα, το σετ χαρακτήρων και το font.

Το `tools/mklng.py` τα κάνει `.LNG`, και το Makefile τα βάζει στη δισκέτα. Τα λατινικά packs μοιράζονται το font `lang/latin1/font.txt`.

5. Studio: προγράμματα με blocks

Το STUDIO είναι editor blocks τύπου Scratch που τρέχει πάνω στον ίδιο τον CPC.

- Φτιάχνεις αντικείμενα (sprites) και scripts από blocks.
- Το πρόγραμμα τρέχει αμέσως με CONTROL+R.
- Γίνεται κανονικό πρόγραμμα του V-OS (`.VBN`) που ξεκινά και από τον Launcher.

Η οθόνη

Το Studio χρησιμοποιεί «μικτή» οθόνη, με δύο ζώνες:

- **Πάνω, σε mode 0 (16 χρώματα):**
  - **αριστερά, η σκηνή** σε μισό μέγεθος
  - **δεξιά, το πάνελ ΑΝΤΙΚΕΙΜΕΝΑ:** τα αντικείμενα με το χρώμα και τη θέση τους, το μέγεθος του επιλεγμένου (W, H), το φόντο και το mode του προγράμματος (π.χ. `ΣΚΗΝΗ 2 M1`), και το όνομα του αρχείου
- **Κάτω, σε mode 1:**
  - **αριστερά, η παλέτα** με τα blocks μιας κατηγορίας (`< ΓΕΓΟΝΟΤΑ >`)
  - **δεξιά, το script**, μία γραμμή για κάθε block
  - **στο τέλος, η γραμμή κατάστασης**, με τα πλήκτρα που ισχύουν εκεί που βρίσκεσαι

Το **TAB** περνά από το script στην παλέτα, μετά στη σκηνή, και πίσω στο script.

Μενού

STUDIO		ΕΠΕΞΕΡΓΑΣΙΑ
ΝΕΟ	^N	ΣΒΗΣΕ ΓΡΑΜΜΗ
ΑΝΟΙΓΜΑ (`.VBS`)	^O	+ ΑΝΤΙΚΕΙΜΕΝΟ (ως 6)
ΑΠΟΘΗΚΕΥΣΗ	^S	− ΑΝΤΙΚΕΙΜΕΝΟ
ΑΠΟΘ. ΩΣ		ΦΟΝΤΟ (το χρώμα της σκηνής)
ΕΚΤΕΛΕΣΗ	^R	MODE ΟΘΟΝΗΣ (0 ή 1)
ΣΤΟ ΠΑΡΑΣΚΗΝΙΟ		
ΕΠΙΣΤΡΟΦΗ	^Q	

- **ΕΚΤΕΛΕΣΗ (^R):** αποθηκεύει το `ONOMA.VBS`, γράφει το `ONOMA.VBN` και το τρέχει. Το πρόγραμμα σταματά από το μενού του (ΠΡΟΓΡΑΜΜΑ > ΤΕΛΟΣ, ^Q), και τότε το Studio ξαναξεκινά με το ίδιο πρόγραμμα ανοιχτό.
- **ΣΤΟ ΠΑΡΑΣΚΗΝΙΟ:** ρωτά αν το πρόγραμμα θα συνεχίζει να τρέχει όταν είναι πίσω από άλλο ή πίσω από μια όψη, ή θα παγώνει, που είναι η προεπιλογή.
- **MODE ΟΘΟΝΗΣ:**
  - **Mode 0:** 16 χρώματα, πλάτος 192 pixels.
  - **Mode 1:** 4 χρώματα (πένες 0–3), πλάτος 384 pixels, και περισσότερο κείμενο.

Η παλέτα

Τα ←/→ αλλάζουν κατηγορία, τα ↑/↓ διαλέγουν block, και το RETURN το προσθέτει κάτω από τη γραμμή του script. Ένα κλικ σε block της παλέτας κάνει το ίδιο.

Ένα block που ανοίγει (ΓΙΑ ΠΑΝΤΑ, ΕΠΑΝΑΛΑΒΕ, ΑΝ) μπαίνει μαζί με το ΤΕΛΟΣ του, και σβήνεται μαζί του. Ένα block με κείμενο ζητά το κείμενο αμέσως.

Το script

Πλήκτρο	Τι κάνει
↑/↓	γραμμή
←/→	πεδίο της γραμμής (τα πεδία είναι οι τιμές μέσα στο block)
RETURN	χωρίς επιλεγμένο πεδίο: προσθέτει το block της παλέτας. Σε πεδίο κειμένου: αλλαγή κειμένου. Σε πεδίο πλήκτρου: περιμένει να πατήσεις το πλήκτρο
CLR	σβήνει τη γραμμή
DEL	σβήνει το τελευταίο ψηφίο ενός αριθμού
ποντίκι	κλικ σε πεδίο: το επιλέγει. Δεύτερο κλικ: αλλάζει το είδος του

Το πρόγραμμα είναι μια λίστα από γραμμές:

- Κάθε `ΟΤΑΝ …` ξεκινά ένα script, για ένα αντικείμενο ή για τη σκηνή. Το script φτάνει ως το επόμενο `ΟΤΑΝ`.
- Η εσοχή βγαίνει μόνη της από τα blocks που ανοίγουν και κλείνουν.
- Γραμμές πριν από το πρώτο `ΟΤΑΝ` αγνοούνται.
- Ό,τι μένει ανοιχτό κλείνει στο τέλος του script.

Τι γράφεται σε ένα πεδίο

Η γραμμή κατάστασης λέει κάθε φορά τι δέχεται το πεδίο.

Πεδίο τιμής:

Πλήκτρο	Τιμή
0–9, −	αριθμός, ακέραιος από −32768 ως 32767
A–H	μεταβλητή. Το ίδιο γράμμα ξανά: A1, A2 … A15, και πάλι A
X, Y	η θέση του αντικειμένου του script
R και ψηφία	ΤΥΧΑΙΟ: ένας αριθμός από 1 ως το n
K και πλήκτρο	ΠΛΗΚΤΡΟ: 1 όσο είναι πατημένο, αλλιώς 0
T και ψηφίο	ΑΓΓΙΖΕΙ: 1 όταν το αντικείμενο αγγίζει το αντικείμενο n
Z	ΣΤΗΝ ΑΚΡΗ: 1 όταν το αντικείμενο είναι στην άκρη της σκηνής
P	ΚΟΥΜΠΙ: ο αριθμός του τελευταίου κουμπιού που πατήθηκε
U	ΚΟΣΤΟΥΜΙ: ο αριθμός του κοστουμιού του αντικειμένου (0: χωρίς κοστούμι)

Άλλα είδη πεδίου:

- **Μεταβλητή:** A–H και πάλι το ίδιο γράμμα για A1…A15.
- **Σύγκριση:** `<`, `=`, `>`. Το SPACE δίνει «καμία», και τότε το ΑΝ ελέγχει μόνο αν η πρώτη τιμή δεν είναι 0.
- **Αντικείμενο** (στα `ΟΤΑΝ`): 0 η σκηνή, 1–6 ένα αντικείμενο.
- **Πλήκτρο** (στο `ΟΤΑΝ ΠΛΗΚΤΡΟ`): RETURN και μετά το πλήκτρο.
- **Κείμενο:** RETURN, και γράφεις ως 14 χαρακτήρες.
- **Ομάδα μεταβλητών** (SAVE/LOAD VARS): A–H προχωρά στην επόμενη οκτάδα (A-H, A1-H1 …

A15-H15), και το SPACE δίνει ΟΛΕΣ.

Τα γράμματα διαβάζονται ως πλήκτρα, οπότε δουλεύουν και με το ελληνικό πληκτρολόγιο.

Η σκηνή και τα αντικείμενα

Πλήκτρο	Τι κάνει
βελάκια	μετακινεί το επιλεγμένο αντικείμενο
1–6	επιλογή αντικειμένου
C	επόμενο χρώμα
W, H	πλάτος, ύψος
K	ποιο κοστούμι φορά (κανένα, 1, 2, ...)
E	σχεδίαση του κοστουμιού του (νέο, αν δεν έχει)
κλικ στη σκηνή	βάζει εκεί το αντικείμενο

Ένα αντικείμενο χωρίς κοστούμι είναι ένα χρωματιστό ορθογώνιο.

Κοστούμια

Το **E** ανοίγει τον επεξεργαστή κοστουμιών στη θέση της σκηνής. Έχει πλέγμα 16×16 και από κάτω τις 16 πένες.

Πλήκτρο	Τι κάνει
βελάκια	κελί
SPACE, RETURN	βάφει το κελί
0–9, A–F	πένα
.	διάφανο
E, TAB	τέλος
κλικ	σε κελί: βάφει. Στη σειρά των πενών: επιλογή πένας

Ένα πρόγραμμα έχει ως 8 κοστούμια, κοινά για όλα τα αντικείμενα. Το μέγεθος ενός αντικειμένου με κοστούμι είναι 16×16. Στη σκηνή, το x στρογγυλεύεται σε ολόκληρο byte (2 pixels σε mode 0, 4 σε mode 1).

Όλα τα blocks

Στη στήλη «Block», τα πεδία φαίνονται όπως στο script.

ΓΕΓΟΝΟΤΑ

Block	Τι κάνει
ΟΤΑΝ ΞΕΚΙΝΑ *αντικ.*	το script ξεκινά μαζί με το πρόγραμμα
ΟΤΑΝ ΠΛΗΚΤΡΟ *πλήκτρο αντικ.*	ξεκινά όταν πατηθεί το πλήκτρο
ΟΤΑΝ ΚΛΙΚ *αντικ.*	ξεκινά με κλικ στο αντικείμενο (ή στη σκηνή)
ΟΤΑΝ ΛΑΒΕΙ *n αντικ.*	ξεκινά όταν σταλεί το μήνυμα n
ΣΤΕΙΛΕ *n*	στέλνει το μήνυμα n (το n μπορεί να είναι και μεταβλητή)

> Ένα μήνυμα ξεκινά **από την αρχή** κάθε script που το περιμένει, ακόμα κι αν έτρεχε ήδη. Γι' αυτό ένα script δεν πρέπει να στέλνει μήνυμα στον εαυτό του.

ΕΛΕΓΧΟΣ

Block	Τι κάνει

ΓΙΑ ΠΑΝΤΑ ... ΤΕΛΟΣ	επαναλαμβάνει για πάντα
ΕΠΑΝΑΛΑΒΕ *n* ... ΤΕΛΟΣ	n φορές
ΑΝ *a σύγκριση b* ... [ΑΛΛΙΩΣ ...] ΤΕΛΟΣ	εκτελεί τις γραμμές μόνο αν ισχύει η σύγκριση
ΠΕΡΙΜΕΝΕ *n*	περιμένει n frames (50 = ένα δευτερόλεπτο)
ΠΕΡΙΜΕΝΕ ΩΣ *a σύγκριση b*	περιμένει ώσπου να ισχύσει η σύγκριση
ΣΤΑΜΑΤΑ	τελειώνει αυτό το script

ΚΙΝΗΣΗ

Block	Τι κάνει
ΠΗΓΑΙΝΕ *x y*	το αντικείμενο πάει στο x, y
ΑΛΛΑΞΕ Χ *n*, ΑΛΛΑΞΕ Υ *n*	μετακινεί κατά n
ΤΑΧΥΤΗΤΑ *vx vy*	ορίζει την ταχύτητά του
ΚΙΝΗΣΟΥ	ένα βήμα με την ταχύτητά του
ΑΝΑΠΗΔΗΣΕ	στην άκρη της σκηνής, αντιστρέφει την ταχύτητα

ΕΜΦΑΝ./ΗΧΟΣ

Block	Τι κάνει
ΧΡΩΜΑ *n*	η πένα του ορθογωνίου
ΚΟΣΤΟΥΜΙ *n*	φορά το κοστούμι n (1–8. Το 0 δεν αλλάζει τίποτα)
ΕΠΟΜΕΝΟ ΚΟΣΤΟΥΜΙ	το επόμενο, και μετά το τελευταίο πάλι το πρώτο
ΕΜΦΑΝΙΣΟΥ, ΚΡΥΨΟΥ	
ΓΡΑΨΕ *τιμή* ΣΤΟ *x y*	τυπώνει έναν αριθμό
ΝΟΤΑ *n* ΓΙΑ *frames*	παίζει νότα (αριθμός MIDI: 60 = ντο, 69 = λα 440 Hz) και περιμένει να τελειώσει

ΜΕΤΑΒΛΗΤΕΣ

Block	Τι κάνει
ΘΕΣΕ *μεταβλητή* ΣΕ *τιμή*	
ΑΛΛΑΞΕ *μεταβλητή* ΚΑΤΑ *τιμή*	πρόσθεση
ΜΕΙΩΣΕ *μεταβλητή* ΚΑΤΑ *τιμή*	αφαίρεση
ΠΟΛΛΑΠΛΑΣΙΑΣΕ *μεταβλητή* ΕΠΙ *τιμή*	
ΔΙΑΙΡΕΣΕ *μεταβλητή* ΔΙΑ *τιμή*	ακέραια διαίρεση
ΥΠΟΛΟΙΠΟ *μεταβλητή* ΔΙΑ *τιμή*	το υπόλοιπο της διαίρεσης
ΑΠΟΘΗΚΕΥΣΕ *ομάδα* ΣΤΟ *"όνομα"*	γράφει τις μεταβλητές σε αρχείο κειμένου (δες παρακάτω)
ΦΟΡΤΩΣΕ *ομάδα* ΑΠΟ *"όνομα"*	τις διαβάζει πίσω

ΠΑΡΑΘΥΡΑ

Block	Τι κάνει
ΠΑΡΑΘΥΡΟ *"τίτλος" πλάτος ύψος*	παράθυρο στη μέση της σκηνής, όπως οι διάλογοι του V-OS. Ένα νέο παράθυρο παίρνει τη θέση του προηγούμενου και των κουμπιών του
ΚΟΥΜΠΙ *"κείμενο"* ΣΤΟ *x y*	κουμπί του συστήματος. Πατιέται με κλικ ή με TAB και RETURN. Αριθμείται από το 0, με τη σειρά που εμφανίζονται
ΟΤΑΝ ΚΟΥΜΠΙ *αντικ.*	ξεκινά όταν πατηθεί οποιοδήποτε κουμπί. Η τιμή ΚΟΥΜΠΙ (Ρ) λέει ποιο
ΚΕΙΜΕΝΟ *"κείμενο"* ΣΤΟ *x y*	γράφει κείμενο

ΕΤΙΚΕΤΑ *κουμπί* ΣΕ *"κείμενο"*	αλλάζει το κείμενο ενός κουμπιού

Με ανοιχτό παράθυρο, τα ΚΕΙΜΕΝΟ, ΚΟΥΜΠΙ και ΓΡΑΨΕ μετρούν από την πάνω αριστερή γωνία του εσωτερικού του. Το παράθυρο σέρνεται από τον τίτλο του, μαζί με τα κουμπιά του.

Μεταβλητές

Υπάρχουν 128 μεταβλητές, κοινές για όλα τα scripts, με ονόματα A–H, A1–H1, ..., A15–H15. Στην αρχή όλες είναι 0, και κρατούν ακέραιους από −32768 ως 32767.

Αποθήκευση και επαναφορά (SAVE VARS, LOAD VARS)

- `ΑΠΟΘΗΚΕΥΣΕ ΟΛΕΣ ΣΤΟ "GAME"` γράφει και τις 128 στο αρχείο `GAME.TXT`. Το `.TXT` μπαίνει όταν το όνομα δεν έχει τύπο.
- Με ομάδα (`A-H`, `A1-H1`, ...) γράφει μόνο εκείνες τις 8.
- Το αρχείο είναι κείμενο, μία γραμμή για κάθε μεταβλητή:

```
A=12 B=0 A1=-5
```

- `ΦΟΡΤΩΣΕ ΟΛΕΣ ΑΠΟ "GAME"` αλλάζει τις μεταβλητές που αναφέρει το αρχείο. Με ομάδα, αλλάζει μόνο όσες είναι και στην ομάδα. Γραμμές που δεν καταλαβαίνει τις αγνοεί, και αν το αρχείο λείπει δεν αλλάζει τίποτα.
- Έτσι ένα αρχείο μπορεί να επαναφέρει όλες τις μεταβλητές ή μόνο μία οκτάδα, π.χ. τις αποθηκευμένες θέσεις ενός παιχνιδιού.

Αρχεία

- **`ΟΝΟΜΑ.VBS`:** το πρόγραμμα όπως το επεξεργάζεσαι. Το ανοίγει το ΑΝΟΙΓΜΑ, και ως 5Κ διαβάζονται.
- **`ΟΝΟΜΑ.VBN`:** το πρόγραμμα έτοιμο να τρέξει. Το γράφει η ΕΚΤΕΛΕΣΗ και ξεκινά και από τον Launcher. Είναι ο παίκτης (`VBRUN.SYS`) μαζί με το πρόγραμμα. Ο παίκτης μεταγλωττίζει τα blocks σε κώδικα Z80 όταν ξεκινά, οπότε τα προγράμματα τρέχουν γρήγορα.

Ένα `.VBS` φτιάχνεται και από κείμενο στον υπολογιστή, με το `tools/mkvbs.py`. Η σύνταξη περιγράφεται στην αρχή του αρχείου, και παραδείγματα είναι τα `blocks/examples/*.vbt` (CALC, MINES, FLAP, TICKER, VARS).

Όρια

Γραμμές (blocks)	240
Αντικείμενα	6
Scripts (`ΟΤΑΝ ...`)	16
Μεταβλητές	128
ΕΠΑΝΑΛΑΒΕ σε ένα πρόγραμμα	64 (από εκεί και πέρα μοιράζονται μετρητή)
Ανοιχτά blocks το ένα μέσα στο άλλο	16
Κείμενα	32, ως 14 χαρακτήρες το καθένα
Κοστούμια	8, 16×16
Κουμπιά	64

Κώδικας	τουλάχιστον 4.000 bytes και για το μεγαλύτερο πρόγραμμα. Ό,τι δεν χωρά αγνοείται

Συμβουλές

- **Scripts της σκηνής:** τα blocks κίνησης και εμφάνισης δεν κάνουν τίποτα, και τα Χ, Υ, ΑΓΓΙΖΕΙ και ΣΤΗΝ ΑΚΡΗ δίνουν 0.
- **ΑΓΓΙΖΕΙ:** ισχύει μόνο όταν φαίνονται και τα δύο αντικείμενα.
- **Συγχρονισμός:** τα scripts τρέχουν «μαζί». Κάθε script παραχωρεί τη σειρά του στο τέλος κάθε επανάληψης και σε κάθε ΠΕΡΙΜΕΝΕ. Για να περιμένει ένα script κάποιο άλλο, χρησιμοποίησε μια μεταβλητή και `ΠΕΡΙΜΕΝΕ ΩΣ` (δες το MINES).
- **Έτοιμα παραδείγματα:** τα CALC, MINES, FLAP και TICKER ανοίγουν στο Studio (ΑΝΟΙΓΜΑ).

6. Blocks στον υπολογιστή: ο web editor και το `vbc.py`

Εκτός από το Studio, που τρέχει στον CPC, υπάρχει και ένας editor blocks για τον υπολογιστή. Η γλώσσα είναι η ίδια, με μερικά blocks παραπάνω: λίστες, θόρυβο, μέγεθος αντικειμένου, κείμενο με χρώμα.

Ο editor

Άνοιξε το `blocks/editor.html` σε browser. Είναι βασισμένος στο Blockly και έχει ελληνικά ή αγγλικά.

- **Κατηγορίες:** γεγονότα, έλεγχος, κίνηση, εμφάνιση, κείμενο, τελεστές, αισθητήρες, μεταβλητές, λίστες και παράθυρα.
- **Αντικείμενα:** ορθογώνια ή κοστούμια. Ορίζονται σε πίνακα, και η σκηνή τα δείχνει με τα χρώματα του CPC. Σέρνονται με το ποντίκι.
- **Αρχεία:** ανοίγει και αποθηκεύει αρχεία `.vbp`. Είναι JSON με όλο το πρόγραμμα, οπότε γράφονται και με το χέρι.

Μεταγλώττιση

```
python3 blocks/vbc.py pong.vbp --dsk build/vos.dsk
```

Το `vbc.py` γράφει το `build/ONOMA.ASM`, με ένα σχόλιο για κάθε block. Το μεταγλωττίζει με τη RASM σε `ONOMA.VBN`, με πίνακα relocation ώστε να τρέχει δίπλα σε άλλα προγράμματα, και με `--dsk` το βάζει στη δισκέτα.

Επιλογή	Τι κάνει
`-o build`	φάκελος εξόδου
`--asm-only`	μόνο το `.ASM`
`--dsk build/vos.dsk`	και στη δισκέτα

Η μορφή `.vbp`

Η πλήρης περιγραφή είναι στην αρχή του `blocks/vbc.py`: γεγονότα, εντολές, τιμές, κοστούμια, εικονίδιο. Σε συντομία:

```
{"name": "PONG", "mode": 0, "background": 0,
 "sprites": [{"name": "ball", "x": 90, "y": 100, "w": 6, "h": 6,
              "colour": 8, "vx": 2, "vy": 2, "visible": true}],
 "variables": {"score": 0},
 "scripts": [{"sprite": "ball", "when": {"event": "start"},
              "do": [{"block": "forever", "do": [{"block": "move"}, {"block": "bounce"}
```

- **Μεταβλητές και λίστες:** έχουν ονόματα, όχι γράμματα. Οι λίστες έχουν σταθερό μέγιστο μήκος.
- **Εικονίδιο:** το `"icon"` δίνει εικονίδιο για τον Launcher.
- **Πρόγραμμα με πολλά όμοια κομμάτια:** μπορεί να το γράψει ένα script σε Python. Δες το `tools/mkbricks.py`, που γράφει το `bricks.vbp` με 32 τούβλα.

Παραδείγματα

Στο `blocks/examples/`:

Αρχείο	Πρόγραμμα
`pong.vbp`	PONG
`catch.vbp`	CATCH
`starcat.vbp`	STARCAT: κοστούμια, μελωδία από λίστα
`counter.vbp`	COUNTER: παράθυρο και κουμπιά
`bricks.vbp`	BRICKS (το γράφει το `tools/mkbricks.py`)

Τα `*.vbt` είναι προγράμματα του Studio σε κείμενο, για το `tools/mkvbs.py`. Βλ. κεφάλαιο 5.

Studio ή web editor;

	Studio	web editor
Πού	στον CPC	στον υπολογιστή
Αρχείο	`.VBS`	`.vbp`
Μεταβλητές	128, A–H15	με ονόματα
Λίστες, θόρυβος, μέγεθος αντικειμένου	όχι	ναι
SAVE/LOAD VARS	ναι	όχι
Το πρόγραμμα	θέλει όλη τη μνήμη	μικρό, τρέχει δίπλα σε άλλα

7. SDK του V-OS: προγράμματα σε assembly

Με αυτό το SDK γράφεις δικά σου προγράμματα σε Z80 assembly. Τρέχουν μέσα στο V-OS όπως το DEMO ή το CLOCK: εμφανίζονται στον Launcher, το Desktop και τον Commander, και έχουν δικό τους μενού, κουμπιά, διαλόγους, ήχο και αρχεία. Τρέχουν και δίπλα σε άλλα προγράμματα.

Γρήγορο ξεκίνημα

```
make new-app NAME=hello
```

Αυτό φτιάχνει τον φάκελο `myapps/hello/` από το `sdk/template/`. Έχει τρία αρχεία:

- `hello.asm`: το πρόγραμμα. Τα βέλη κινούν ένα κουτί, υπάρχει ένα κουμπί ΗΧΟΣ, ένας μετρητής δευτερολέπτων, και ένα δικό του μενού (ABOUT, BACK) δίπλα στα VIEW και HELP.
- `texts.txt`: τα κείμενά του, σε αγγλικά και ελληνικά.
- `icon.txt`: το εικονίδιό του, 16×16, για τον Launcher.

```
make app NAME=hello
```

Αυτό χτίζει το `build/HELLO.BIN` και το βάζει στη δισκέτα `build/vos.dsk`.

```
make run
```

Αυτό ανοίγει τη δισκέτα στον Caprice32. Το HELLO είναι στον Launcher, στην τελευταία σελίδα.

- Το απλό `make` χτίζει και βάζει στη δισκέτα όλα τα προγράμματα του `myapps/`, μαζί με το V-OS.
- Το όνομα έχει 1–8 πεζά γράμματα ή ψηφία και ξεκινά με γράμμα. Δεν μπορεί να είναι όνομα προγράμματος του V-OS (π.χ. demo).
- Χρειάζεσαι τη RASM, Python 3 και το iDSK, όπως για όλο το V-OS.
- Στον Caprice32 το πρόγραμμα το τρέχεις εσύ, από τον Launcher.

Μεγαλύτερο πρόγραμμα:

- Βάλε κι άλλα `.asm` στον φάκελό του (π.χ. `art.asm` με sprites) και φέρε τα με `include "art.asm"`. Το build τα βρίσκει, και ξαναχτίζει το πρόγραμμα όταν αλλάζουν.
- Ό,τι βάλεις στον υποφάκελο `disc/` (π.χ. `disc/LEVEL1.DAT`) μπαίνει ως έχει στη δισκέτα, για το `VOS_LOAD`.
- Με υποφάκελο `owndisc/`, το πρόγραμμα παίρνει και δική του δισκέτα, `build/<όνομα>.dsk`, με το `.BIN` και τα αρχεία του `owndisc/`. Τη βάζεις στο B: και το τρέχεις από τον Commander (ΑΡΧΕΙΟ > ΕΚΚΙΝΗΣΗ). Από ποιον οδηγό ξεκίνησε, για να φορτώσει τα αρχεία του από εκεί, το λέει το `VOS_SELF`: `db οδηγός`, και μετά το όνομά του (11 bytes).
- Ένα πρόγραμμα με πίνακα relocation (όπως αυτά του SDK) χωράει ως 12K. Για τα υπόλοιπα, φόρτωσε αρχεία στη bank δεδομένων.

Πώς δουλεύει ένα πρόγραμμα

Ένα πρόγραμμα είναι ένα `.BIN`, χτισμένο για το `VOS_APP_ADDR` (`#4000`). Ξεκινά έτσι:

```
        include "api/vos_api.inc"
        org VOS_APP_ADDR
```

```
        jp init                 ; +0
        jp event                ; +3  HL -> το γεγονός
        jp redraw               ; +6
        db "VA", VOS_API_VERSION
        include "app_icon.asm"  ; +12 το εικονίδιο (προαιρετικό)
```

- **`init`** ορίζει mode, παλέτα, μενού και κουμπιά, ανοίγει τα `VE_FRAME` αν χρειάζεται, και σχεδιάζει τα πάντα.
  - A = 0: νέα εκκίνηση.
  - A = 2: το πρόγραμμα επιστρέφει από μια όψη ή άλλο πρόγραμμα. Η κατάστασή του είναι όπως την άφησε· ορίζει ξανά τα παραπάνω και σχεδιάζει.
- **`event`** καλείται για κάθε γεγονός και επιστρέφει αμέσως. Ένα πρόγραμμα δεν περιμένει ποτέ μόνο του (κανένας βρόχος αναμονής). Για κίνηση ή χρόνο, `VOS_TICKS` με A = 1, και τότε έρχεται ένα `VE_FRAME` 50 φορές το δευτερόλεπτο.
- **`redraw`** ξανασχεδιάζει τα πάντα, π.χ. όταν αλλάζει η γλώσσα ή κλείνει το desk accessory.
- **Έξοδος:** `VOS_EXIT`, που επιστρέφει στην όψη που το ξεκίνησε.

Η σειρά μέσα στο `event`

```
event:  ld a,(hl)
        cp VE_FRAME : jp z,frame        ; τα frames
        push hl
        call VOS_UI_EVENT               ; μενού, κουμπιά, CONTROL+γράμμα
        pop hl
        or a : jr nz,command            ; A = η εντολή
        call VOS_MENU_OPEN : ret nz     ; ανοιχτό μενού: τα πλήκτρα είναι δικά του
        ...                             ; τα δικά σου πλήκτρα και κλικ
command: ...
        jp VOS_COMMAND                  ; ό,τι δεν είναι δικό σου (VIEW, HELP)
```

Όσο είναι ανοιχτό μενού, μη σχεδιάζεις. Αλλιώς θα γράψεις πάνω του. Στο `VE_FRAME`, ενημέρωσε την κατάσταση, αλλά σχεδίασε μόνο αν το `VOS_MENU_OPEN` δίνει Z. Οι διάλογοι (`VOS_DIALOG`, `VOS_INPUT`, `VOS_FILE_PICK`) έχουν δικό τους βρόχο, οπότε δεν έρχεται κανένα γεγονός όσο είναι ανοιχτοί.

Τα γεγονότα

`HL` → 8 bytes: `db type, code, char, mods; dw x, y`.

type	τι	code / char / x, y
`VE_KEY_DOWN` 1, `VE_KEY_REPEAT` 3, `VE_KEY_UP` 2	πλήκτρο	code = αριθμός πλήκτρου (`VK_*`, γραμμή·8 + bit του πληκτρολογίου), char = ο χαρακτήρας στη γλώσσα του pack
`VE_PTR_DOWN` 5, `VE_PTR_UP` 6, `VE_PTR_MOVE` 4	δείκτης	x, y σε pixels του mode
`VE_FRAME` 8	frame (με `VOS_TICKS`)	—
`VE_LANG` 7	άλλαξε η γλώσσα του πληκτρολογίου	—

Τρία είδη προγράμματος

- **Με πίνακα relocation (το προεπιλεγμένο του SDK):** το `make app` το χτίζει δύο φορές, στο `#4000` και στο `#4100`. Το `tools/mkreloc.py` προσθέτει τον πίνακα από τη διαφορά.

    - Τρέχει δίπλα σε άλλα προγράμματα, στα 12K από το `#5000`, και συνεχίζει όπου σταμάτησε (με F6 φαίνονται τα προγράμματα που τρέχουν).
    - Τα δεδομένα του πρέπει να είναι μέσα στα δικά του bytes (labels, `ds`), ποτέ σε σταθερές διευθύνσεις.
    - Αν το `mkreloc.py` βγάλει σφάλμα, κάπου χρησιμοποιείται διεύθυνση με τρόπο που δεν αντέχει μετακίνηση. Π.χ. `ld a,label & 255`, ή μόνο το low byte μιας διεύθυνσης.

- **Χωρίς πίνακα:** παίρνει όλη τη bank (ως 16K, κώδικας και δεδομένα), αλλά τρέχει μόνο του, εκτός αν υπάρχει επέκταση μνήμης. Για αυτό χρειάζεται δικός σου κανόνας στο Makefile (βλ. `STUDIO.BIN`).
- **Desk accessory:** ως 4K, με πίνακα. Ανοίγει με F5 πάνω από οτιδήποτε, και το `init` παίρνει A = 1. Βλ. `apps/clock/clock.asm` και την περιγραφή στο `src/api/vos_api.inc`.

Κανόνες

- **Καταχωρητές:** οι κλήσεις χαλάνε όλους τους καταχωρητές εκτός από το IX. Τα ορίσματα μπαίνουν σε A, BC, DE, HL (IX για `VOS_RECT` και `VOS_BLIT`). Τα αποτελέσματα επιστρέφουν σε A ή HL. Σε αποτυχία γυρίζει carry και A = αριθμός σφάλματος, που τον δείχνει το `VOS_ERROR`.
- **Μνήμη:** μην απλώνεις άλλη bank στο `#4000`, γιατί το πρόγραμμα θα εξαφανιστεί από κάτω σου. Για 16K επιπλέον υπάρχει η bank δεδομένων (`VOS_BANK_GET`, `VOS_BANK_PUT`, `VOS_DATA`). Είναι κοινή με όψεις και άλλα προγράμματα, και κρατά τα δεδομένα της μόνο όσο τρέχει το πρόγραμμα.
- **Συντεταγμένες:** x σε pixels του mode (mode 0: 0–191, mode 1: 0–383), y από 0 ως 258. Η γραμμή μενού πιάνει τα y 0–11.
- **RASM:** οι ετικέτες δεν ξεχωρίζουν κεφαλαία από πεζά (`BOX` και `box` είναι το ίδιο). Τα hex γράφονται με `#` και όχι με `&`.

Οι μορφές των δεδομένων

- **Γραμμή μενού** (`VOS_SET_BAR`): `db πλήθος; dw μενού, ...`. Τα κοινά VIEW και HELP τα δίνουν τα `VOS_MENU_VIEW` και `VOS_MENU_HELP`.
- **Μενού:** `db τίτλος, πλήθος` και για κάθε επιλογή `db κείμενο, εντολή, γράμμα, 0`.
    - κείμενο: `VOS_SEPARATOR` για γραμμή
    - εντολή: 1.., ό,τι επιστρέφει το `VOS_UI_EVENT`
    - γράμμα: συντόμευση CONTROL+γράμμα, ή 0
- **Κουμπιά** (`VOS_SET_WIDGETS`, μετά `VOS_DRAW_WIDGETS`): `db πλήθος`, και για κάθε κουμπί `dw x, y; db πλάτος σε χαρακτήρες, κείμενο, εντολή, 0`.
- **Διάλογος** (`VOS_DIALOG` → A = κουμπί, 0 το πρώτο, το ESC δίνει το τελευταίο): `db τίτλος, κείμενο, πλήθος κουμπιών (1-3), κουμπί, ...`. Για κουμπιά, `VM_OK`, `VM_YES` και `VM_NO`.
- **Κείμενα** (`VOS_TEXTS`): τα φτιάχνει το `tools/mktexts.py` από το `texts.txt`, με γραμμές `ONOMA | English | Ελληνικά`.
    - Δίνει `T_ONOMA` (`VT_FIRST` = `#C0` και πάνω) και τον πίνακα `texts:`. Ως 64 κείμενα.
    - Το `|` χωρίς κενά γύρω του αλλάζει γραμμή στους διαλόγους.
    - Τα κείμενα μπαίνουν όπου ζητείται «κείμενο» (μενού, κουμπιά, διάλογοι). Με το `VOS_MSG` παίρνεις τη διεύθυνσή τους για `VOS_PRINT`.

- **Παλέτα** (`VOS_PALETTE`): 17 χρώματα firmware (0–26): 16 πένες και το περίγραμμα. Στο mode 1 μετράνε οι πένες 0–3.
- **Ορθογώνιο** (`VOS_RECT`, IX →): `dw x, y, w, h; db πένα`.
- **Εικονίδιο:** 16 γραμμές των 16 χαρακτήρων στο `icon.txt`. Κάθε χαρακτήρας είναι πένα σε hex (mode 0, παλέτα του Launcher) ή `.` για διάφανο.
- **Ονόματα αρχείων** (`VOS_LOAD`, `VOS_SAVE`, `VOS_DELETE`, `VOS_RUN`): 11 bytes, `"NAME EXT"`. Drive 0 = A:, 1 = B:.

Το API (έκδοση 11)

Κλήση	Είσοδος → έξοδος	Τι κάνει
`VOS_EXIT`	—	τέλος, πίσω στην όψη που το ξεκίνησε
`VOS_TICKS`	A = 1 / 0	`VE_FRAME` 50 φορές το δευτερόλεπτο, ή κανένα
`VOS_FRAMES`	→ HL (και A τα bits 16–23)	frames από την εκκίνηση
`VOS_RANDOM`	→ A	0–255
`VOS_COMMAND`	A = εντολή	οι κοινές εντολές των VIEW και HELP
`VOS_ERROR`	A = σφάλμα	διάλογος σφάλματος
`VOS_MODE`	A = 0 / 1	mode 0 (192×259, 16 πένες) ή 1 (384×259, 4 πένες)
`VOS_PALETTE`	HL → 17 χρώματα	
`VOS_CLS`	A = πένα	καθαρίζει την οθόνη
`VOS_PLOT`	HL = x, DE = y, A = πένα	ένα pixel
`VOS_RECT`	IX → ορθογώνιο	γεμάτο ορθογώνιο
`VOS_COLOURS`	A = πένα κειμένου, C = φόντου	για τα `VOS_PRINT` και `VOS_CHAR`
`VOS_PRINT`	HL = x, DE = y, BC → κείμενο, 0	
`VOS_CHAR`	HL = x, DE = y, A = χαρακτήρας	
`VOS_MSG`	A = κείμενο → HL	η διεύθυνση ενός κειμένου (`T_*` ή του συστήματος)
`VOS_SET_BAR`, `VOS_DRAW_BAR`	HL → γραμμή μενού	
`VOS_UI_EVENT`	(το τρέχον γεγονός) → A	εντολή μενού, κουμπιού ή συντόμευσης, 0 για καμία
`VOS_DIALOG`	HL → διάλογος → A	
`VOS_INPUT`	A = τίτλος, HL → κείμενο, C = μέγιστο → carry αν ακυρώθηκε	εισαγωγή κειμένου (χρειάζεται C + 2 bytes)
`VOS_SET_WIDGETS`, `VOS_DRAW_WIDGETS`	HL → κουμπιά	
`VOS_MENU_VIEW`, `VOS_MENU_HELP`	→ HL	τα κοινά μενού
`VOS_CATALOG`	A = drive → A = αρχεία, HL → λίστα	16 bytes το καθένα: 11 το όνομα, στο +12 το μέγεθος σε K
`VOS_FILE_PICK`	A = τίτλος → A = αρχείο, carry αν ακυρώθηκε	διάλογος επιλογής, μετά το `VOS_CATALOG`
`VOS_LOAD`	A = drive, HL → όνομα, DE = πού → BC = μήκος	

VOS_SAVE	A = drive, HL → όνομα, DE = από, BC = μήκος	αρχείο binary, που αντικαθιστά το παλιό
VOS_DELETE	A = drive, HL → όνομα	
VOS_TEXTS	HL → πίνακας κειμένων	τα κείμενα του προγράμματος
VOS_MENU_OPEN	→ NZ αν είναι ανοιχτό μενού	
VOS_KEY_HELD	A = πλήκτρο → NZ όσο πατιέται	για παιχνίδια
VOS_BLIT	IX → `dw x, y; db bytes, γραμμές; dw πηγή; db βήμα πηγής`	εικόνα: ζεύγη (μάσκα, pixels)
VOS_SOUND	A = κανάλι 0–2, HL = περίοδος (bit 15: θόρυβος), C = ένταση 0–15, B = frames	νότα· το 440 Hz είναι περίοδος 142
VOS_SPLIT	HL → `dw y1, y2; db 8 χρώματα` (0 = σβήνει)	μικτή οθόνη: mode 0 ανάμεσα στα y1 και y2
VOS_RUN	A = drive, HL → όνομα	τρέχει άλλο πρόγραμμα, και μετά ξεκινά πάλι αυτό
VOS_KEEP	→ HL → 16 bytes	μνήμη που μένει ανάμεσα σε προγράμματα
VOS_BANK_GET, VOS_BANK_PUT	HL = από, DE = προς, BC = bytes	αντιγραφή από και προς τη bank δεδομένων
VOS_BACKGROUND	A = 1 / 0	συνεχίζει ή παγώνει όταν είναι πίσω από άλλο
VOS_DRAG	HL = x, DE = y, BC = w, A = h → carry: νέα HL, DE	σύρσιμο παραθύρου από τον τίτλο του
VOS_FIT	ίδια → HL, DE	φέρνει ένα παράθυρο μέσα στην οθόνη

Πλήρεις λεπτομέρειες, π.χ. για το παρασκήνιο, το accessory και το `VOS_SPLIT`, έχει το `src/api/vos_api.inc`. Για παραδείγματα, δες τα `apps/demo/demo.asm` (mode 0, κίνηση), `apps/clock/clock.asm` (accessory) και `apps/mixed/mixed.asm` (μικτή οθόνη). Ένα ολόκληρο παιχνίδι με sprites, πίστα από αρχείο και ήχο είναι το `myapps/vrouves/`.

Έλεγχος

Το `make test-sdk` χτίζει το πρότυπο ως TEMPLATE.BIN και το ανοίγει από τον Launcher στον Caprice32, με πατήματα πλήκτρων που ορίζει το `src/kernel/test_script.asm` (σενάριο 15). Τραβά screenshots στο `build/shots`: την οθόνη του, το κουτί αφού κινηθεί, το μενού του, τον διάλογο ABOUT, και την επιστροφή στον Launcher.

Για δικό σου αυτόματο test, αντέγραψε το σενάριο 15 και τον στόχο `test-sdk` του Makefile.

8. Ανάπτυξη του V-OS

Για όσους δουλεύουν στο ίδιο το V-OS. Τα προγράμματα *για* το V-OS περιγράφονται στο SDK.

Τι χρειάζεται

- RASM, ο assembler
- iDSK, για τις εικόνες δισκέτας
- Python 3, για τα εργαλεία. Για το `make docs` χρειάζεται και τα `markdown` και `pymupdf`.
- GNU make 4.3 ή νεότερο
- για δοκιμές, ο Caprice32 (snap `caprice32`)

Build

Εντολή	Τι κάνει
`make`	`build/vos.dsk`: ο kernel, οι όψεις, οι γλώσσες, όλα τα προγράμματα και τα `myapps/*`
`make run`	ανοίγει τη δισκέτα στον Caprice32 (`RUN"VOS"`)
`make shot`	εκκίνηση χωρίς παράθυρο και ένα screenshot
`make docs`	το manual ως PDF: `docs/V-OS.pdf` (ελληνικά) και `docs/V-OS-en.pdf`, `-fr`, `-es`, `-de`, `-it`
`make new-app NAME=x`, `make app NAME=x`	δικό σου πρόγραμμα (SDK)
`make clean`	σβήνει το `build/`

Δομή του repo

Φάκελος	Περιεχόμενο
`src/vos.asm`	η ρίζα του build: loader και kernel, εγγραφή στη δισκέτα
`src/boot/`	ο loader που τρέχει η AMSDOS στο `#4000`, ανίχνευση επέκτασης RAM
`src/kernel/`	vectors, interrupts, σύστημα αρχείων, όψεις, API (`api.asm`), ρυθμίσεις, kernel extension (`ext.asm`), σενάρια δοκιμών (`test_script.asm`)
`src/drivers/`	Gate Array, CRTC, πληκτρολόγιο, ποντίκι AMX, FDC, AY
`src/gfx/`	γραφικά overscan, κείμενο, μικτή οθόνη
`src/ui/`	μενού, κουμπιά, διάλογοι, εισαγωγή κειμένου, επιλογή αρχείου
`src/shell/`	οι όψεις ως overlays: Desktop, Launcher, Commander, Setup
`src/api/vos_api.inc`	το API για τα προγράμματα
`lang/`	οι γλώσσες (κείμενα, πληκτρολόγια, fonts)
`apps/`	τα προγράμματα σε assembly: DEMO, MIXED, STUDIO, CLOCK, και τα εικονίδιά τους
`blocks/`	ο web editor, το `vbc.py`, το `runtime.asm`, ο παίκτης του Studio (`player.asm`), παραδείγματα
`sdk/`	το SDK: οδηγός και πρότυπο προγράμματος
`tools/`	εργαλεία: fonts, γλώσσες, κείμενα, εικονίδια, relocation, `.VBS` από κείμενο, screenshots, PDF

`docs/`	αυτή η τεκμηρίωση στα ελληνικά (`manual/`) και στα αγγλικά, γαλλικά, ισπανικά, γερμανικά, ιταλικά (`manual/en/`, `fr/`, `es/`, `de/`, `it/`), τα PDF, οι απαιτήσεις (`REQUIREMENTS.md`)

Η μνήμη, σε συντομία

Περιοχή	Τι έχει
`#0000–#3DFF`	ο kernel. Στη σελίδα μηδέν, μετά τα restart vectors, ο πίνακας του API στο `#0108`
ως `#4000`	η στοίβα του kernel
`#4000–#7FFF`	το «παράθυρο» των banks
`#8000–#FFFF`	η οθόνη overscan 32K. Στα κενά της, buffers του kernel

Οι banks που μπαίνουν στο παράθυρο:

Bank	Τι έχει
4	buffer αρχείων και φόρτωσης, και η bank δεδομένων του προγράμματος που τρέχει
5	η όψη στο `#4000–#4FFF`, και από το `#5000` τα προγράμματα με πίνακα relocation (ή ένα πρόγραμμα χωρίς πίνακα σε όλη τη bank)
6	ό,τι κρύβουν τα μενού και οι διάλογοι, και από το `#7000` το desk accessory
7	το language pack, οι λίστες του Commander, τα αγγλικά κείμενα, και από το `#7000` το kernel extension

Με επέκταση μνήμης, οι banks της παίρνουν προγράμματα. Οι λεπτομέρειες, διεύθυνση προς διεύθυνση, είναι στο `docs/REQUIREMENTS.md` (§4.2 και σε κάθε «Υλοποίηση Μ...»).

Συμβάσεις

- **RASM:** τα hex γράφονται `#C000` και όχι `&C000`, γιατί η RASM διαβάζει το `&` ως AND. Οι ετικέτες δεν ξεχωρίζουν κεφαλαία από πεζά.
- **Σχόλια:** στα αγγλικά, σύντομα, όπως στον υπάρχοντα κώδικα. Κάθε αρχείο ξεκινά με ένα μπλοκ που λέει τι κάνει.
- **Τεκμηρίωση:** στα ελληνικά. Το `README.md` και το `docs/REQUIREMENTS.md` έχουν μια ενότητα για κάθε milestone. Το manual γράφεται στα ελληνικά και μεταφράζεται στις άλλες πέντε γλώσσες του V-OS.
- **Κείμενα:** τα κείμενα του συστήματος είναι στο `lang/*/messages.txt`, και των προγραμμάτων σε `texts.txt` (για το `tools/mktexts.py`).

Δοκιμές

Οι δοκιμές τρέχουν το V-OS στον Caprice32 χωρίς παράθυρο και τραβούν screenshots στο `build/shots/<καθυστέρηση>/`.

Πώς δουλεύουν:

- Ο Caprice32 χωρίς παράθυρο δεν μπορεί να πατήσει βελάκια, COPY ή πλήκτρα f. Γι' αυτό τα πατήματα τα κάνει ένα ειδικό build: ένα `-DTEST_SCRIPT=n` βάζει στο `src/kernel/test_script.asm` το σενάριο n, έναν πίνακα με πλήκτρα ανά frame.
- Κάθε στόχος `make test-…` χτίζει μια δισκέτα με το σενάριό του και τραβά ένα screenshot σε κάθε καθυστέρηση του `…_SHOTS`.
- Η καθυστέρηση μετριέται σε πατήματα «Z»: περίπου 2,5–3,5 frames το καθένα. Η αντιστοιχία δεν είναι σταθερή, οπότε ένα στιγμιότυπο το πιάνεις με δύο-τρεις καθυστερήσεις γύρω του.

- Αφού το V-OS γράψει στη δισκέτα, ο Caprice32 δεν κλείνει μόνος του, και η δοκιμή τελειώνει με timeout. Είναι αναμενόμενο σε test-vars, test-costume κ.ά.
- Τα screenshots τα κοιτάζεις, π.χ. συνδυασμένα σε μία εικόνα με `ffmpeg … hstack/vstack`.

Στόχος	Τι δοκιμάζει
`test-input`	πλήκτρα, Studio: blocks, αρχεία, εκτέλεση
`test-calc`	η CALC με κλικ, μετά το `CALC.VBS` στο Studio
`test-costume`	κοστούμι στο Studio, εκτέλεση, ξανά άνοιγμα
`test-acc`	το ρολόι ως desk accessory
`test-multi`, `test-switch`, `test-expand`, `test-behind`	πολλά προγράμματα, F6, επέκταση μνήμης, παρασκήνιο
`test-drag`	σύρσιμο παραθύρων
`test-langs`, `test-lang`	οι γλώσσες, η αποθήκευση της γλώσσας
`test-mines`, `test-bricks`	τα παιχνίδια
`test-vars`	οι 128 μεταβλητές, SAVE/LOAD VARS
`test-sdk`	το πρότυπο του SDK
`test-vrouves`	το VROUVES από τη δισκέτα του στο B:, μέσω Commander: χτίσιμο, το κύμα 1, τα λεφτά
`test-amsdos`, `test-reboot`, `test-system`	τα αρχεία του V-OS στη BASIC, επανεκκίνηση, δισκέτα SYSTEM

Νέο σενάριο:

1. Πρόσθεσε ένα `IF TEST_SCRIPT == n` στην αρχή της αλυσίδας του `test_script.asm`, με `ELSE` πριν από το επόμενο και ένα ακόμα `ENDIF` στο τέλος.
2. Πρόσθεσε έναν στόχο στο Makefile, όπως το `test-sdk`.

Κάθε πλήκτρο γράφεται ως `dw από, ως : db γραμμή, bit` του πληκτρολογίου. Ο αριθμός του πλήκτρου είναι γραμμή·8 + bit, π.χ. RETURN = 18 = γραμμή 2, bit 2.

Χωρίς εξομοιωτή: κώδικας ενός προγράμματος τρέχει και σε Z80 μέσα στην Python (πακέτο `z80`). Φορτώνεις το `.BIN` στο `#4000` και βάζεις τις κλήσεις του API να επιστρέφουν αμέσως (ή να κάνουν ό,τι χρειάζεται, π.χ. αρχεία σε λεξικό). Τότε καλείς το init και το event και διαβάζεις τη μνήμη. Είναι πολύ γρηγορότερο από τα screenshots για λάθη σε αριθμούς, αρχεία και άλματα. Τις διευθύνσεις των labels τις δίνει το `rasm … -s -sl` (αρχείο `.sym`).

Κάθε milestone

1. Κώδικας, και δοκιμή στον Caprice32: νέα ή υπάρχουσα, με screenshots.
2. Οι δοκιμές που αγγίζει η αλλαγή.
3. Ενότητα στο `README.md` και «Υλοποίηση M…» στο `docs/REQUIREMENTS.md`.
4. **Ενημέρωση του manual και στις έξι γλώσσες** (`docs/manual/` στα ελληνικά, `docs/manual/<κωδικός>/` για en, fr, es, de, it, και τα `sdk/README.md`, `sdk/README.<κωδικός>.md` αν αλλάζει το API), και `make docs` για όλα τα PDF.
5. Commit και push.